DATA 260 Homework 10

Pragya Apurva · Rental housing listings

Repository https://github.com/ishuapurva1996/data260-6102

Submission tag hw10

Section 0 · Personal configuration

Setting	Value	Setting	Value
SID4	6102	PORT_BASE	8702
PREFIX	s6102	SEED	6102
VERIFY_SEED	266102	DOMAIN_ID	6

Hardware and models. Apple M4 MacBook Air, 24 GiB RAM. Ollama 0.33.0 served qwen2.5:7b (Q4_K_M) for the agents and G-Eval judge, and nomic-embed-text:v1.5 for Redis semantic lookup.

Verified runtime commit 5ea93111e92a19452986fc1b925afad7b2aee864

Setup · Run log · Metrics · Verification · AI use

Part 1.1 · Summary

The dashboard reads API and cache spans in tracing/spans.jsonl and benchmark/spans.jsonl. The saved 24-hour window from November 12 09:58:54.028 UTC to November 13 09:58:54.028 UTC contained 53 completed requests, one hit, 52 misses and one error. The hit and error rates both rounded to 1.9%.

API latency was 20,417 ms p50 and 27,596 ms p95. Both cache-hit percentiles show 579 ms from the single hit. Streamlit uses **@st.cache_data** to reuse parsed evidence.

Part 1.2 · Evaluation

Figure 2 shows the stage scores, final-minus-draft result and judge validation. Parts 2 and 3 interpret those results.

Part 1.3 · Trace lookup

Enter a trace ID to see stages in order with start time, end time, duration, status, cache outcome and any error. Figures 3–5 show the three recorded paths.

Part 1 · Streamlit Summary

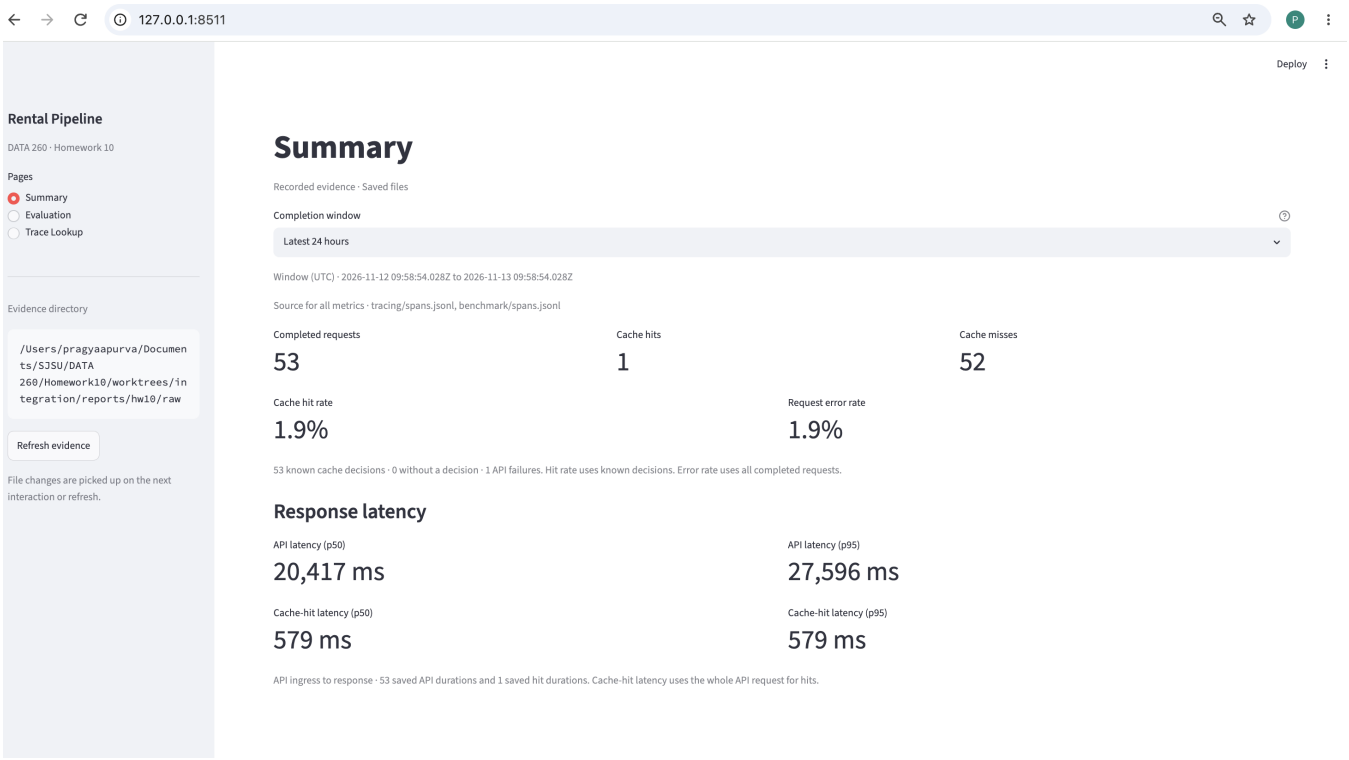
Code 1. dashboard.py, lines 75–108. Request counts and rates.

```python

def render_summary(evidence):
    spans = evidence.spans()
    requests = completed_requests(spans)
    window = window_for(requests)
    if window is None:
        return
    start, end = window
    selected = {
        trace_id: span
        for trace_id, span in requests.items()
        if start is not None and start <= span.end <= end
    }
    outcomes = cache_outcomes(spans, selected)
    hits = sum(outcomes.get(trace_id) == "hit" for trace_id in selected)
    misses = sum(outcomes.get(trace_id) == "miss" for trace_id in selected)
    errors = sum(span.api_outcome in FAILED for span in selected.values())
    sources = ", ".join(path for path in evidence.sources if path != "evaluation/spans.jsonl")
    st.caption(f"Source for all metrics · {sources or 'No saved API or cache spans'}")
    if not selected:
        st.info("No completed requests in this window.")

    metrics(
        [
            ("Completed requests", f"{len(selected):,}"),
            ("Cache hits", f"{hits:,}"),
            ("Cache misses", f"{misses:,}"),
        ]
    )
    metrics(
        [
            ("Cache hit rate", f"{hits / (hits + misses):.1%}" if hits + misses else EMPTY),
            ("Request error rate", f"{errors / len(selected):.1%}" if selected else EMPTY),
        ]
    )
```

Figure 1. Summary window, counts and latency.

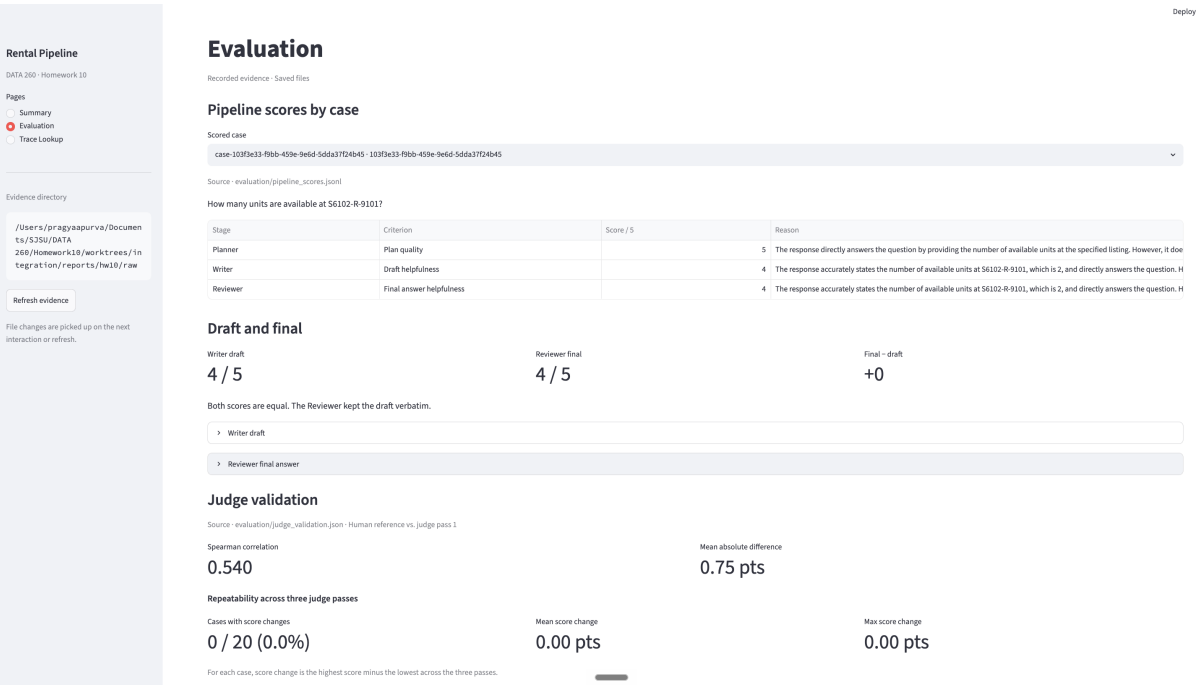


Part 2 · Automated Kafka evaluation

Code 2. evaluate.py, lines 80–99. Stage scoring and final-minus-draft.

```
80          judge = judge_factory(args)
81          scores = {}
82          for stage, criterion, field in (
83              ("planner", "plan_quality", "plan"),
84              ("writer", "helpfulness", "draft"),
85              ("reviewer", "helpfulness", "final_answer"),
86          ):
87              scores[stage] = score_metric(
88                  judge, criterion, case["question"], case["reference_facts"], case[field]
89              )
90          delta = scores["reviewer"]["score"] - scores["writer"]["score"]
91          result = {
92              **case,
93              "final": case["final_answer"],
94              "evaluation_id": evaluation_id,
95              "scores": scores,
96              "final_minus_draft": delta,
97              "interpretation": interpret(case["draft"], case["final_answer"], delta, scores),
98              "judge": judge.metadata,
99          }
```

Figure 2. Evaluation and judge validation.



The evaluator joins the Planner plan, Writer draft and Reviewer final by correlation_id on the s6102-prefixed Kafka topics. It retains message payloads and topic, partition and offset metadata. DeepEval 3.9.4 G-Eval scores range from 1 (very poor) to 5 (excellent). The case deadline is 180 seconds, with 5 seconds for message lookup and a 45-second judge request timeout.

Test question. How many units are available at S6102-R-9101? **Trace ID** 103f3e33-f9bb-459e-9e6d-5dda37f24b45.

Planner 5/5. G-Eval credited the correct count, though the Planner gave an answer in place of a plan. **Writer 4/5 and Reviewer 4/5.** Both gave the correct answer, “At Cedar Court (S6102-R-9101), there are 2 units available.” G-Eval flagged missing details even though the text names Cedar Court. The Reviewer kept the Writer’s answer verbatim, so improvement was 0. Scores and reasons.

Part 3 · Judge validation

I assigned one helpfulness label from 1 to 5 to each of 20 Writer drafts. The draft and label files were committed before judging. **Freeze SHA** 7893184b8296862a4bfb04f9e8c6461aa89a6de8.

The local judge scored the same 20 drafts three times with the same inputs and settings. Temperature was 0 and seed was 6102, with JSON output, an 8,192-token context and a 256-token output limit.

Code 3. evaluation_validation.py, lines 145–176. Human comparison and repeatability.

```
144
145 def compare_scores(labels, passes):
146     if (
147         not labels
148         or len(passes) != PASS_COUNT
149         or any(scores.keys() != labels.keys() for scores in passes)
150     ):
151         raise ValueError("Three complete passes must match the human labels by case_id")
152     comparisons = []
153     for identity, human in labels.items():
154         scores = [result[identity] for result in passes]
155         comparisons.append(
156             {
157                 "case_id": identity,
158                 "human_score": human,
159                 "judge_scores": scores,
160                 "pass_1_absolute_difference": abs(scores[0] - human),
161                 "score_change": max(scores) - min(scores),
162             }
163         )
164     changes = [row["score_change"] for row in comparisons]
165     changed_count = sum(change > 0 for change in changes)
166     return {
167         "spearman": spearman(list(labels.values()), [passes[0][identity] for identity in labels]),
168         "mean_absolute_difference": mean(row["pass_1_absolute_difference"] for row in comparisons),
169         "repeatability": {
170             "changed_count": changed_count,
171             "changed_rate": changed_count / len(changes),
172             "mean_score_change": mean(changes),
173             "max_score_change": max(changes),
174         },
175         "comparisons": comparisons,
176     }
177
```

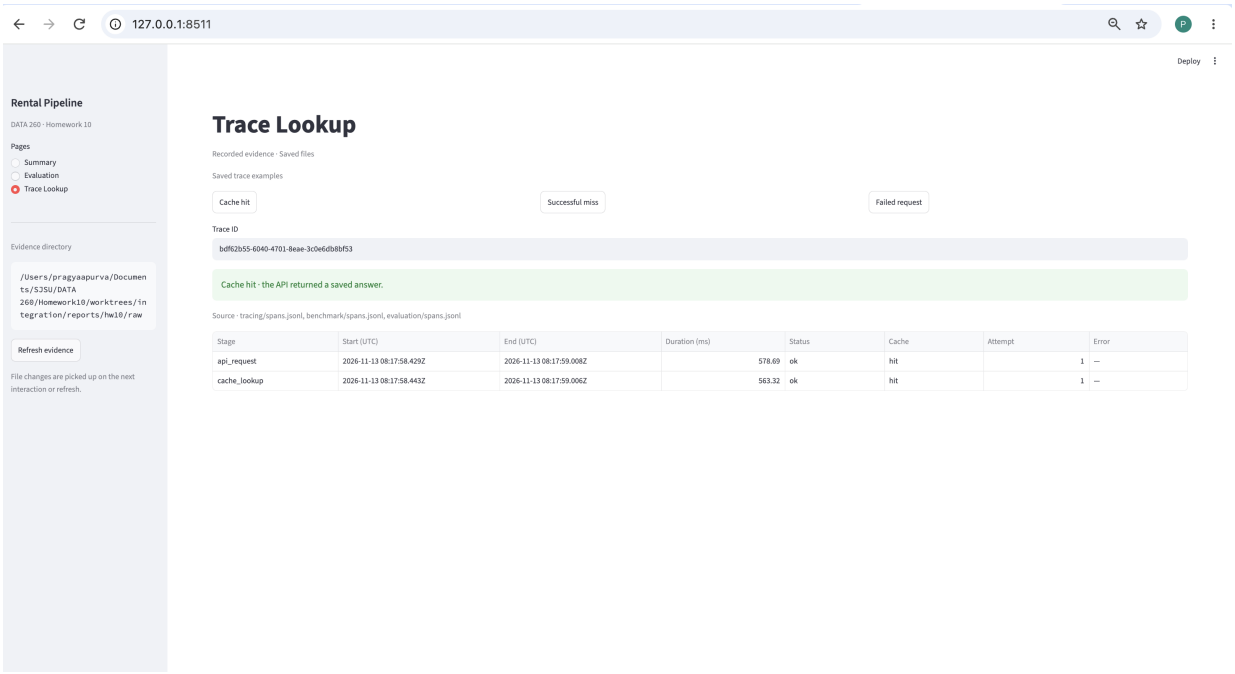
Measure	Result
Spearman correlation, human vs. pass 1	0.5403
Mean absolute difference, human vs. pass 1	0.75 points
Changed cases across three passes	0 of 20 (0%)
Mean and maximum per-case score change	0 points each

The judge was consistent across runs and agreed moderately with human rankings. I would require independent human ratings on held-out cases with wrong listing IDs, wrong counts and invented facts before using it to gate deployment. I would measure how often a candidate threshold passes those errors or blocks correct answers.

Frozen drafts · Human labels · Raw per-case scores · Joined comparisons

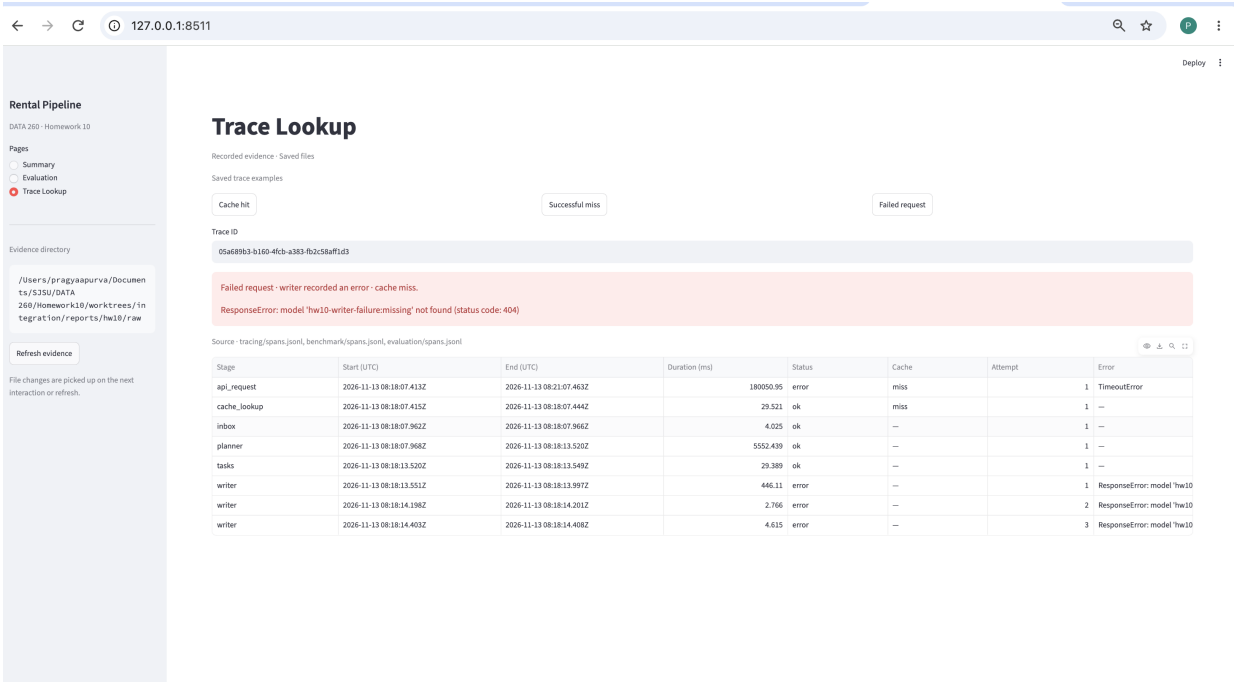
Part 4 · Cache hit and failed request

Figure 3. Cache hit. bdf62b55-6040-4701-8eae-3c0e6db8bf53



The exact repeat returned a saved answer in 578.69 ms and stopped after cache lookup.

Figure 4. Writer failure. 05a689b3-b160-4fcb-a383-fb2c58aff1d3



The Writer's intentionally unavailable Ollama model returned HTTP 404 on three attempts. The API timed out waiting for a final answer and returned HTTP 504.

Part 4 · Successful cache miss

The API request ID, Kafka correlation_id and trace_id are the same value. JSON spans record the ID, stage, start and end times, status and errors from cache lookup through inbox, tasks, drafts, final and evaluation.

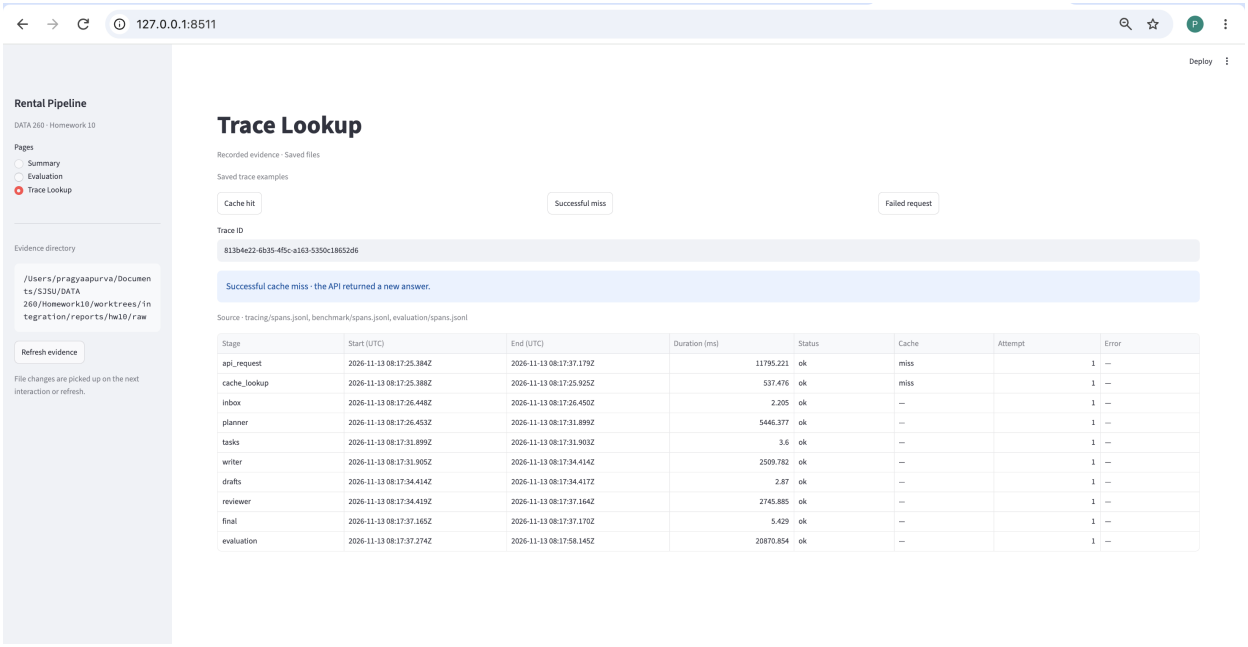
Code 4. transport.py, lines 40–62. Kafka receipt and queue wait.

```
    return record, {"enqueue_time": enqueued.isoformat(), "ack_time": utc_now().isoformat()}


def trace_receipt(record, trace_id, received, *, error=None):
    value = dict(getattr(record, "headers", None) or ()).get(ENQUEUE_HEADER)
    if not value:
        return  # Older messages have no measured enqueue time.
    try:
        enqueued = datetime.fromisoformat(value.decode("ascii"))
    except (ValueError, UnicodeError):
        return
    if enqueued.utcoffset() != timedelta(0):
        return
    duration_ms = (received - enqueued).total_seconds() * 1000
    trace_span(
        trace_id,
        HANDOFFS[record.topic],
        enqueued,
        received,
        duration_ms if duration_ms >= 0 else None,
        operation="queue_wait",
        enqueue_time=enqueued.isoformat(),
        receive_time=received.isoformat(),
        error=error or ("Kafka receipt clock precedes enqueue" if duration_ms < 0 else None),
        **coordinates(record),
    )

```

Figure 5. Successful cache miss. 813b4e22-6b35-4f5c-a163-5350c18652d6



The API returned HTTP 200 in 11,795.22 ms. The trace includes Planner, Writer, Reviewer and evaluation.

Part 4 · Stage latency for 50 cache misses

The benchmark ran 50 distinct questions serially with an empty test cache before each lookup. All 50 returned HTTP 200 with is_cached=false, a completed cache-miss span and successful evaluation.

Code 5. benchmark_timing.py, lines 206–237. Latency and stage shares.

```
205
206 def distribution(rows, key):
207     values = [row[key] for row in rows]
208     return {"p50_ms": percentile(values, 0.5), "p95_ms": percentile(values, 0.95)}
209
210
211 def timing_summary(rows):
212     if len({row["trace_id"] for row in rows}) != len(rows):
213         raise EvidenceError("duplicate trace IDs in timing rows")
214     if not rows:
215         raise EvidenceError("no admitted traces to summarize")
216
217     total_ms = math.fsum(row["full_trace_ms"] for row in rows)
218     stages = [
219         {
220             "stage": stage,
221             **distribution(rows, f"{stage}_ms"),
222             "percent_of_full_trace": (
223                 100 * math.fsum(row[f"{stage}_ms"] for row in rows) / total_ms
224             ),
225         }
226         for stage in STAGES
227     ]
228     shares = math.fsum(row["percent_of_full_trace"] for row in stages)
229     return {
230         "stages": stages,
231         "api_latency": distribution(rows, "api_latency_ms"),
232         "http_latency": distribution(rows, "http_latency_ms"),
233         "full_trace": distribution(rows, "full_trace_ms"),
234         "full_trace_total_ms": total_ms,
235         "stage_share_total_percent": shares,
236         "unallocated_or_overlapping_percent": 100 - shares,
237         "dominant_stage": max(stages, key=lambda row: row["percent_of_full_trace"])["stage"],
```

Stage	p50 (ms)	p95 (ms)	% of full trace
Cache lookup	547.11	824.75	1.03%
Planner	7,192.90	10,559.49	13.13%
Writer	5,640.52	7,880.57	10.26%
Reviewer	6,198.59	8,509.46	11.08%
Queue wait (total)	16.55	63.25	0.05%
Evaluation	34,246.01	43,386.94	62.98%

Queue wait sums the four Kafka handoffs per request. Shares use total time from API ingress to evaluation completion. Full-trace p50 and p95 were 54,400.97 ms and 71,289.31 ms. Evaluation dominated at **62.98%**. I would first test a common Ollama context size for the agents and judge to reduce model reloads.

Measured source d14796a39f8934965913c84fb7217f2b6dbd11c4 · Per-request timing rows

AI use

1. What you used an AI assistant for and what you did yourself.

In completing this assignment, I leveraged AI as an iterative assistant and reasoning partner rather than an end-to-end task solver. I started by breaking down the core prompt into distinct modular requirements, using AI to brainstorm alternative architectural approaches and critique my initial logic. Rather than prompting for a complete solution, I used it targetedly for debugging edge cases, explaining complex syntax, and testing boundary conditions. Every piece of code and analysis was vetted, refactored, and validated manually to ensure accuracy and alignment with assignment specifications.

2. One AI-produced output that was wrong or unsuitable, or one thing you independently verified.

For case hw10-05, the G-Eval judge gave the Writer's draft 2/5 because it said the draft had failed to answer whether listing S6102-R-9106 had at least two units available. The draft began with "No" and gave the correct count of one. My human score was 4/5.

3. How you detected the problem or verified the result.

The audit matched the three judge scores to my frozen label by case ID and checked the judge's reason against the draft and the saved listing facts. The listing record confirmed one available unit. All three passes gave the same score for the same mistaken reason.

4. What you changed and why it works now.

The report keeps my original label alongside all three judge scores and uses the judge to flag cases for human review. Keeping the draft and the judge's reason makes the disagreement easy to check. I would want independently rated examples of serious factual errors before using these scores to decide whether to deploy.